PARTITUNE

PROCESADORES DE LENGUAJES

Curso 2024/2025

APELLIDOS, NOMBRES	PUNTUACIÓN
BALLESTEROS MORALLÓN, ELENA	5
BURGOS ORTEGA, JORGE	5
CHINARRO CABRERO, MARCELO	5
ESPEJO GIL, SAMUEL	5

INDICE

1. PRESENTACIÓN DEL PROBLEMA 2
1.1. Estado del Arte 2
1.2. Implementación final 3
1.3. Justificación del uso de los conocimientos de la materia 3
2. EBFN 5
3. DESCRIPCIÓN SEMÁNTICA DEL LENGUAJE 7
4. DIAGRAMAS EN T 10
5. ANALIZADOR LÉXICO 11
5.1. Tabla de tokens 11
5.2. EBNF Sintáctico 11
6. NOTACIÓN MUSICAL ABC 13

1. PRESENTACIÓN DEL PROBLEMA

En el ámbito de la composición musical, existen diversas herramientas software que permiten la creación de partituras de complejidades muy variadas. No obstante, una de las principales limitaciones de estas herramientas es que aprender a utilizarlas o dominar los lenguajes que emplean, requiere de una considerable inversión de tiempo y esfuerzo. Algunos aspectos generales son:

- El uso de interfaces gráficas hace muy tedioso el proceso, ya que arrastrar elementos y realizar ajustes precisos no siempre es lo más eficiente.
- Aunque algunos lenguajes de programación permiten crear partituras complejas, esto a menudo sacrifica la rapidez y simplicidad que muchos usuarios buscan.

1.1. Estado del Arte

Existen otros proyectos que abarcan lenguajes musicales. Los más notables son:

- https://github.com/AustinKobayashi/music-dsl: Proyecto de javascript que crea partituras. Especifica el EBNF. Tiene una parte de definición de secciones que contienen notas a dibujar y su duración respecto al tempo de la sección. Al final cuenta con un apartado para indicar qué secciones serán mostradas. En general, es un lenguaje simple, con pocas funciones y, además, verboso al necesitar especificar la duración para todas las notas.
- https://github.com/antoinebqt/Music-DSL: Permite tocar canciones describiendo secuencias de notas dando también la duración de cada una. Permite crear repeticiones mediante variables que guardan listas de notas y es posible volver a usarlas, incluso indicando cantidad de veces repetidas. Permite varios instrumentos. Está escrito en java y usa ANTLR para reconocer el lenguaje.
- https://abcnotation.com/: Muestra una representación gráfica y otra sonora, es un estándar muy bien documentado. Es el más completo ya que permite muchas estructuras diferentes y se puede añadir letra para las canciones. Esto también dificulta la lectura de la partitura y hace que sea más compleja, aunque el lenguaje es muy cercano a las representaciones gráficas. Las notas se asumen que duran una longitud determinada a no ser que se especifique lo contrario con un número.

Todas estas herramientas en general requieren repetir muchos elementos que quizá se podrían agrupar.

El proyecto **PartiTune** nace para resolver estos problemas, ofreciendo una solución enfocada en la **rapidez y simplicidad** para la creación de **partituras musicales** de **complejidad no muy exigente**. A través de un **lenguaje formal intuitivo**, los usuarios podrán escribir partituras de manera más ágil y directa, sin las complicaciones que implican los softwares actuales. A modo de resumen, PartiTune sacrifica la creación de composiciones musicales complejas a cambio de proporcionar una manera más rápida

y natural de crear composiciones más simples. Además, este lenguaje está pensado para ser utilizado por usuario con conocimiento medio y en español.

1.2. Implementación final

Se diseñará una aplicación que permita al usuario introducir una partitura en lenguaje PartiTune y le dará la representación sonara y visualmente en una partitura musical. Para esto, se usará uno de los lenguajes descritos anteriormente, ABC.

Este lenguaje es el más utilizado para escribir aplicaciones musicales y cuenta con varias librerías. De esta forma, PartiTune genera un archivo intermedio con formato abc. Sin embargo, las principales diferencias son:

- En ABC hay que escribir las líneas divisorias para separar los compases, PartiTune los calcula automáticamente.
- En general, la escritura de PartiTune es más rápida y sencilla de aprender, por ejemplo, resulta más cómodo escribir varias notas de la misma duración juntas, en vez de escribir en cada nota su duración, que es menos intuitivo y está más alejado de la escritura tradicional de partituras musicales.
- PartiTune está en español, tanto los elementos del lenguaje como los elementos musicales, por ejemplo, las notas mientras que ABC usa la nomenclatura anglosajona

1.3. Justificación del uso de los conocimientos de la materia

Este proyecto requiere aplicar una serie de conceptos fundamentales de la asignatura “Procesadores de Lenguajes”. Los aspectos clave que justifican el uso de estos conocimientos son:

1. **Creación del compilador:** Gracias a esta asignatura, hemos podido dividir el problema que se planteaba en partes más pequeñas, que corresponden con las partes de un compilador. Así, hemos trabajado de forma iterativa e incremental lo que nos ha facilitado el trabajo como programadores, con un diseño organizado y claro.
2. **Diseño del Lenguaje:** Para la creación del lenguaje utilizamos gramáticas EBNF, que facilitan el diseño y especificación de lenguajes. Esta gramática es la base de nuestro trabajo, por lo que un buen diseño es fundamental.
3. **Análisis Léxico:** Para procesar correctamente el lenguaje musical, es necesario reconocer cuáles son los componentes léxicos y saber reconocerlos mediante expresiones regulares para poder distinguirlos de los componentes sintácticos.
4. **Análisis Sintáctico:** Hemos utilizado dos tipos de analizadores: descendente, con ANTLR y ascendente, con CUP. Los conocimientos de la asignatura nos han ayudado a resolver los conflictos entre producciones eficientemente, ya que comprendemos el funcionamiento interno.

5. **Análisis Semántico y Generación de Código:** Antes de convertir un árbol sintáctico con la representación en PartiTune a texto en ABC, es necesario comprobar que se cumplen algunas restricciones que no se han podido tratar ni de forma léxica ni sintáctica (por ejemplo, las claves). Mientras realizamos estas comprobaciones, se generan objetos intermedios que finalmente se traducen a texto ABC.
6. **Tratamiento de Errores:** Es fundamental que PartiTune sea robusto y amigable para el usuario, lo que implica gestionar los errores léxicos y sintácticos de forma clara y efectiva señalando la línea y la columna. Además, el sistema es capaz de recuperarse de los errores que no son críticos y continuar con el análisis. Esto mejora la experiencia del usuario al corregir errores de manera ágil, sin romper el flujo de trabajo.
7. **Optimización y usabilidad:** Se han aplicado principios de optimización para ayudar al usuario en la generación de partituras. Además, el diseño del lenguaje busca minimizar la curva de aprendizaje, priorizando la claridad y la simplicidad para el usuario.

2. EBFN

A continuación, se presenta la descripción EBNF del lenguaje de PartiTune:

```
PARTITURA       ::= { FRAGMENTO } OBRA;
CADENA          ::= {string | num | caracteres};
STRING          ::= letter{letter};
NUM             ::= digit{digit};
CARACTERES      ::= “.” | “,” | “!” | “¡” | “?” | “¿” | “&” | “(“ | “)”;
COMENTARIOS ::= “/*” cadena “*/”;
ID              ::= letter {letter | digit };

OBRA ::= “obra” “ " ” CADENA “ " ” “{“ {ESPECIFICACION} BLOQUE_NOTAS “}”;

ESPECIFICACION::= [CLAVE] | [COMPAS] | [ARMADURA];
CLAVE           ::= “clave” “=” NOTA_BASE;
COMPAS          ::= “compas” “=” COMPAS_DURACION;
ARMADURA        ::= “armadura” “=” NOTAS_ARMADURA;
COMPAS_DURACION ::= "2/4" | "3/4" | "4/4" | "6/8";
NOTAS_ARMADURA ::= MODIFICADOR_ARMADURA NOTA_BASE
                   { MODIFICADOR_ARMADURA NOTA_BASE } ;

FRAGMENTO       ::= “fragmento” ID “{“ {ESPECIFICACION} BLOQUE_NOTAS “}” ;
BLOQUE_NOTAS ::= “{” ELEMENTO { ELEMENTO } “}” ;
ELEMENTO        ::= REPETICION | SECUENCIA | LIGADURA | LLAMADA;
PRINCIPIO_REP   ::= “|” “:” ;
FINAL_REP       ::= “:” “|” ;
REPETICION      ::= [ PRINCIPIO_REP ] SECUENCIA FINAL_REP ;
LLAMADA         ::= ID;
LIGADURA        ::= "lig" "{" SECUENCIA_NOTAS {SECUENCIA_NOTAS} "}" ;
SECUENCIA       ::= DURACION "(" (SILENCIO | NOTA) {SILENCIO | NOTA} ")" ;
DURACION        ::= "sf" | "f" | "sc" | "c" | "n" | "bl" | "r" ;
SILENCIO        ::= " - ";

NOTA            ::= [ MODIFICADOR ] NOTA_BASE [ OCTAVA ] [ PUNTILLO ] ;
```

```
NOTA_BASE   ::= "DO" | "RE" | "MI" | "FA" | "SOL" | "LA" | "SI" ;
MODIFICADOR_ARMADURA ::= "b" | "#";
MODIFICADOR ::= MODIFICADOR_ARMADURA | "%";
OCTAVA          ::= " ' " | " _ " ;
PUNTILLO        ::= "." ;
```

Ejemplo de fichero de entrada

```
/* Declaracion de fragmentos*/

/* define un fragmento cuyo nombre es “simple” */
fragmento simple {
   /* no incluye atribuciones */
   {
   /* bloque de notas a tocar */
      n(FA)
   }
}

fragmento intro {
    /* incluye atribuciones a compás y armadura */
    compas = 2/4
    armadura = b SOL b SOL
    compas = 3/4
    {
        n(RE' MI') /* secuencia */
        simple /* llamada al fragmento simple */
        lig { sc (FA' SOL' LA') } /* ligadura */
        |: n (DO' RE' MI') c (FA' %SOL') :| /* repetición */
    }
}

/* creación de la obra con nombre “Las Cuatro Estaciones” */
obra “Las Cuatro Estaciones” {
      compas = 3/4
      clave = SOL
      {
           intro
           f (DO' RE' MI') sc (FA' SOL' LA')
      }
}
```

3. DESCRIPCIÓN SEMÁNTICA DEL LENGUAJE

Restricciones generales:

- No se tienen en cuenta las mayúsculas y minúsculas, excepto en el título de la obra.

A continuación, se definen las restricciones de cada elemento del lenguaje.

PARTITURA ::= { FRAGMENTO } OBRA;

Restricciones:

- Los fragmentos son opcionales, pero normalmente necesarios para incluir contenido musical evitando la repetición y facilitando la escritura. Los fragmentos deben aparecer antes del bloque principal de obra.
- Los comentarios pueden encontrarse en cualquier punto del documento.
- Si encontramos dos fragmentos con el mismo nombre cogeremos uno cualquiera ya que entraremos en un estado incorrecto, por tanto no se generará nada.

OBRA ::= “obra” “ " ” CADENA “ " ” “{“ {ESPECIFICACION} BLOQUE_NOTAS “}”;

Restricciones:

- Cadena representa el título de la obra, tiene que estar escrito entre comillas.
- Si no se define ningún valor en especificación se usarán los valores más comunes, que por defecto son:
    - clave = SOL
    - compas = 2/4
    - armadura = vacía, ningún elemento.

FRAGMENTO ::= “fragmento” ID “{“ {ESPECIFICACION} BLOQUE_NOTAS “}” ;

Restricciones:

- El ID del fragmento debe ser único. Si ya existe el ID, el usuario recibirá un error.
- Si no se especifican los atributos de compás, clave y armadura, se usarán los mismos que se hayan definido en el bloque global.

ESPECIFICACION ::= [CLAVE] | [COMPAS] | [ARMADURA];

Las especificaciones indican qué valores tendrá los diferentes atributos de la partitura como la clave, el compás o la armadura.

Restricciones:

- Da igual el orden en el que se escriban estos atributos.

- Son opcionales tanto en los fragmentos como en la obra.
- Si se definen varios valores para un mismo atributo, se usará solo el último de ellos. El usuario recibirá un aviso.

CLAVE ::= “clave” “=” NOTA_BASE;

Restricciones:

- Solo se admiten las notas: SOL, FA o DO, sin ningún modificador.

ARMADURA ::= “armadura” “=” NOTAS_ARMADURA;

Representa la armadura o las alteraciones aplicadas a las notas clave. Se hará como una lista separada por espacios de notas modificadas.

Restricciones:

- Como mínimo, debe haber una nota en la armadura.
- Si se definen varios modificadores para la misma nota y octava, sólo se usará la última de estas. El usuario recibirá un aviso. Por ejemplo:

armadura = b SOL b SOL ---> solo se cuenta una vez

- Los modificadores deben ser bemoles ("b") o sostenidos ("#").
- No se pueden mezclar los tipos de modificador en una misma armadura.
- ABC limita las armaduras (ver apartado 6)

ELEMENTO ::= REPETICION | SECUENCIA | LIGADURA | LLAMADA;

Define los elementos a tocar dentro de un fragmento o la obra. Los elementos son repeticiones, secuencias de notas, ligaduras o fragmentos.

Restricciones:

- Las llamadas deben ser identificadores de fragmentos definidos.

REPETICION ::= [PRINCIPIO_REP] SECUENCIA FINAL_REP ;

Restricciones:

- Siempre que aparezca un símbolo de inicio debe aparecer el de fin, si no se considerará un error.
- Puede aparecer un signo de fin de repetición sin que aparezca uno de inicio, por lo que se repite la pieza musical desde el principio o hasta que se encuentre otro símbolo de inicio de repetición.

LIGADURA ::= "lig" "{" SECUENCIA_NOTAS {SECUENCIA_NOTAS} "}" ;

Restricciones:

- Debe haber al menos dos notas en la ligadura.
- La secuencia de notas no puede contener silencios.

SECUENCIA ::= DURACION "(" (SILENCIO | NOTA) { SILENCIO | NOTA} ")" ;

Restricciones:

- Los elementos, silencios o notas, estarán separados por espacios en blanco, tabuladores y saltos de línea.

DURACION ::= "sf" | "f" | "sc" | "c" | "n" | "bl" | "r" ;

Especifica la duración de las notas. El significado de cada símbolo es: semifusa (sf), fusa (f), semicorchea (sc), corchea (c), negra (n), blanca (bl) y redonda (r).

NOTA ::= [MODIFICADOR] NOTA_BASE [OCTAVA] [PUNTILLO] ;

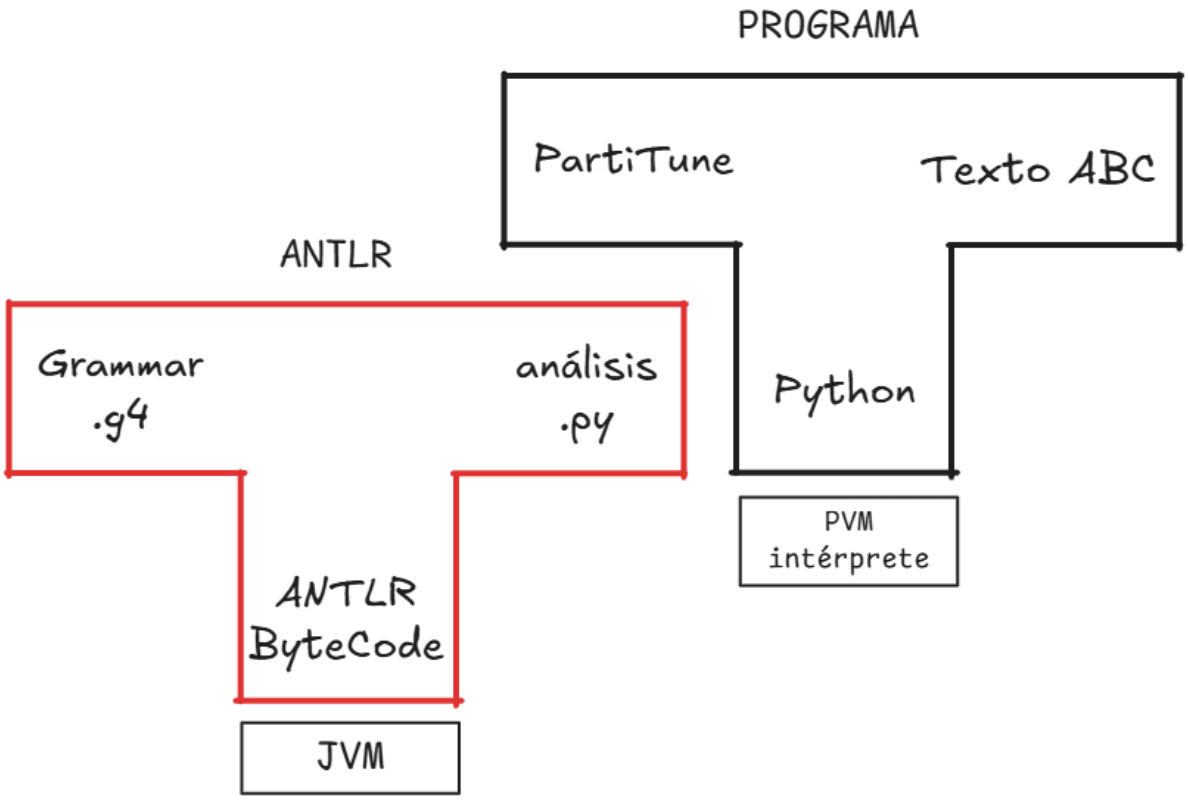
Define una nota musical que puede tener un modificador (sostenido, bemol o becuadro), una nota base, una octava y un puntillo (para prolongar su duración).

Restricciones:

- El modificador es opcional. El becuadro se usará sólo si existe armadura para la nota, si se diese el caso contrario, se daría una advertencia al usuario, al igual que si se usa otro modificador mientras se usa una armadura.
- La octava es opcional y puede ser ascendente (') o descendente (_).
- Es **obligatorio** que el usuario escriba **espacios** entre los modificadores, las notas y las octavas. Por ejemplo: b SOL

4. DIAGRAMAS EN T

PartiTune, permite definir partituras en forma de texto plano. Una vez escrita la partitura, se pasa a notación ABC y con herramientas externas podremos crear una partitura gráfica a partir del texto, otra posibilidad es generar un archivo de audio. Para ello se hace uso de tecnologías como Python y ANTLR, podemos entender los pasos necesarios haciendo uso de los diagramas en T:



ANTLR usa la máquina virtual de Java para ejecutar y crear archivos de código fuente de Python que reconocen el léxico y sintaxis de PartiTune. Esta gramática ha sido definida en un archivo g4 usando el formato que ANTLR requiere.

A este reconocedor en Python se le añaden las partes semánticas, de generación y control de errores. Desarrolladas también en código fuente Python. Se necesitarán ciertas librerías de Python como el runtime de ANTLR y otras para procesar los fragmentos y evitar grafos cíclicos de llamadas.

Una vez se tiene todo esto, el programa se empaquetará, el usuario necesita tener un intérprete de Python instalado para poder ejecutarlo. La salida del programa será la partitura en formato ABC, en el proyecto también se incluye una interfaz web que conecta PartiTune con editores de ABC que permiten visualizar la partitura y reproducir el audio.

5. ANALIZADOR LÉXICO

5.1. Tabla de tokens

Token	Lexema	Patrón
FRAGMENTO	fragmento	fragmento
OBRA	obra	obra
ID	Intro_1	[a-zA-Z] [a-zA-Z|0-9]*
TITULO	“Sonata Nro2.”	“ [^”]* “
CLAVE	clave	clave
COMPAS	compas	compas
ARMADURA	armadura	armadura
COMPAS_DURACION	2/4	2/4 | 3/4 | 4/4 | 6/8
BEMOL	b	b
SOSTENIDO	#	#
NOTA_BASE	RE	DO | RE | MI | FA | SOL | LA | SI
OCTAVA	_	' | _
PUNTILLO	.	\.
PRINCIPIO_REP	|:	|:
FINAL_REP	:|	:|
LIGADURA	lig	lig
DURACION	sc	sf | f | sc | c | n | bl | r
SILENCIO	-	-
BECUADRO	%	%
LLAVE_IZQ	{	{
LLAVE_DER	}	}
PARENTESIS_IZQ	(	(
PARENTESIS_DER	)	)
IGUAL	=	=

5.2. EBNF Sintáctico

```
PARTITURA ::= { FRAGMENTO } OBRA;

OBRA        ::= obra  titulo  llave_izq  {ESPECIFICACION}  BLOQUE_NOTAS  llave_der;


ESPECIFICACION ::= [CLAVE] | [COMPAS] | [ARMADURA];

CLAVE              ::= clave igual nota_base;

COMPAS             ::= compas igual compas_duracion;

ARMADURA           ::= armadura igual NOTAS_ARMADURA;

NOTAS_ARMADURA ::= (bemol | sostenido) nota_base  { (bemol | sostenido)  nota_base } ;


FRAGMENTO ::= fragmento  id  llave_izq  {ESPECIFICACION}  BLOQUE_NOTAS  llave_der;
```

```
BLOQUE_NOTAS ::= llave_izq ELEMENTO { ELEMENTO } llave_der;
ELEMENTO        ::= REPETICION | SECUENCIA | LIGADURA | LLAMADA;
REPETICION      ::= [ principio_rep ] SECUENCIA {SECUENCIA}  final_rep ;
LLAMADA         ::= id;
LIGADURA        ::= ligadura llave_izq SECUENCIA {SECUENCIA} llave_der;

SECUENCIA  ::= duracion parentesis_izq (silencio| nota) { silencio | nota } parentesis_der ;

MODIFICADOR ::= bemol | sostenido | becuadro;
NOTA ::=  [MODIFICADOR] nota_base [octava] [puntillo] ;
```

Cabe destacar que, a la hora de hacer la implementación con ANTLR, hubo que modificar la semántica debido al siguiente caso:

```
Scanner
BEMOL: 'b';
NOTA_BASE: 'do' | 're' | 'mi' | 'fa' | 'sol' | 'la' | 'si';
ID: [a-z]+;

Salida
b --> token BEMOL
sol --> token NOTA_BASE
bsol --> token ID
```

El analizador era capaz de reconocer los tokens de bemol y nota_base, pero al estar juntos no reconoce dos tokens diferentes si no que devuelve el token id. Esto obliga al usuario a escribir el bemol siempre separado de la nota, y no como originalmente se diseñó el lenguaje que es escribiendo todo junto.

6. NOTACIÓN MUSICAL ABC

Como mencionamos anteriormente, utilizaremos el lenguaje ABC para el apartado de generación de nuestro sistema. La decisión de emplear ABC se fundamenta en varias ventajas:

En primer lugar, su sintaxis es minimalista y estandarizada, lo que simplifica el desarrollo de un traductor eficiente entre nuestro lenguaje PartiTune y el formato ABC.

En segundo lugar, el ecosistema que rodea a ABC cuenta con una variedad de aplicaciones que permiten generar partituras impresas, reproducir música o convertir las partituras a otros formatos. Estas capacidades nos permiten centrar los esfuerzos de implementación en la mejora y simplificación de nuestro lenguaje, aprovechando las herramientas ya disponibles. A continuación, describimos los elementos comunes entre ambos lenguajes:

Correspondencia entre las notas musicales

En el sistema de notación ABC, las notas musicales se representan mediante las letras anglosajonas. A continuación, mostramos la correspondencia entre las notas utilizadas en la notación tradicional y su equivalente en el sistema anglosajón:

- Do: C
- Re: D
- Mi: E
- Fa: F
- Sol: G
- La: A
- Si: B

Esta correspondencia es fundamental para entender cómo se estructuran las melodías en el lenguaje ABC.

Estructura Básica del Lenguaje ABC

Un archivo de ABC está compuesto por dos partes principales:

1. **Cabecera**: Contiene metadatos sobre la pieza musical. Algunos de los elementos comunes son:
    - **X**: Número de referencia.
    - **T**: Título de la pieza.
    - **M**: Compás (*por ejemplo, 4/4 o 3/4*).

- **L**: Duración de la nota más corta (*por ejemplo, 1/8 para corcheas*).
- **K**: Tonalidad, es decir, clave y armadura (*por ejemplo, C para Do mayor*).

2. **Cuerpo**: Contiene las notas de la melodía. En esta sección, las notas se escriben en líneas de texto utilizando letras anglosajonas, separadores de compases y símbolos de repetición (*Ejemplo: C D E F | G A B C | C E G C ||*)
    - Los símbolos "|" separan los compases (en ABC son opcionales)
    - El símbolo "||" es el cierre final.
    - Los símbolos de repetición son ":|" y su inverso "|:".

Graves y Agudos

Se indican con comas ',' para notas graves y con apóstrofes ' para notas agudas. Por ejemplo:

- C, (Do grave).
- C' (Do agudo).

Alteraciones:

- Bemol: _ (por ejemplo, _B para Si bemol).
- Sostenido: ^ (por ejemplo, ^F para Fa sostenido).
- Becuadro: = (por ejemplo, =E para Mi becuadro).

Armaduras

En ABC, las armaduras se definen utilizando la tonalidad especificada en la cabecera (campo K). Aunque no se pueden crear armaduras personalizadas, se utilizan las preestablecidas por convenio musical:

Clave	Número de bemoles/sostenidos	ABC Notation	Notas afectadas
Clave de Sol	Sin armadura	K:C	Ninguna
	1 bemol	K:F	Bb
	2 bemoles	K:Bb	Bb, Eb
	3 bemoles	K:Eb	Bb, Eb, Ab
	4 bemoles	K:Ab	Bb, Eb, Ab, Db
	5 bemoles	K:Db	Bb, Eb, Ab, Db, Gb
	6 bemoles	K:Gb	Bb, Eb, Ab, Db, Gb, Cb
	7 bemoles	K:Cb	Bb, Eb, Ab, Db, Gb, Cb, Fb
	1 sostenido	K:G	F#
	2 sostenidos	K:D	F#, C#

	3 sostenidos	K:A	F#, C#, G#
	4 sostenidos	K:E	F#, C#, G#, D#
	5 sostenidos	K:B	F#, C#, G#, D#, A#
	6 sostenidos	K:F#	F#, C#, G#, D#, A#, E#
	7 sostenidos	K:C#	F#, C#, G#, D#, A#, E#, B#
Clave de Fa	Sin armadura	K:C clef=bass	Ninguna
	1 bemol	K:F clef=bass	Bb
	2 bemoles	K:Bb clef=bass	Bb, Eb
	3 bemoles	K:Eb clef=bass	Bb, Eb, Ab
	4 bemoles	K:Ab clef=bass	Bb, Eb, Ab, Db
	5 bemoles	K:Db clef=bass	Bb, Eb, Ab, Db, Gb
	6 bemoles	K:Gb clef=bass	Bb, Eb, Ab, Db, Gb, Cb
	7 bemoles	K:Cb clef=bass	Bb, Eb, Ab, Db, Gb, Cb, Fb
	1 sostenido	K:G clef=bass	F#
	2 sostenidos	K:D clef=bass	F#, C#
	3 sostenidos	K:A clef=bass	F#, C#, G#
	4 sostenidos	K:E clef=bass	F#, C#, G#, D#
	5 sostenidos	K:B clef=bass	F#, C#, G#, D#, A#
	6 sostenidos	K:F# clef=bass	F#, C#, G#, D#, A#, E#
	7 sostenidos	K:C# clef=bass	F#, C#, G#, D#, A#, E#, B#
Clave de Do	Sin armadura	K:C clef=alto	Ninguna
	1 bemol	K:F clef=alto	Bb
	2 bemoles	K:Bb clef=alto	Bb, Eb
	3 bemoles	K:Eb clef=alto	Bb, Eb, Ab
	4 bemoles	K:Ab clef=alto	Bb, Eb, Ab, Db
	5 bemoles	K:Db clef=alto	Bb, Eb, Ab, Db, Gb
	6 bemoles	K:Gb clef=alto	Bb, Eb, Ab, Db, Gb, Cb
	7 bemoles	K:Cb clef=alto	Bb, Eb, Ab, Db, Gb, Cb, Fb
	1 sostenido	K:G clef=alto	F#
	2 sostenidos	K:D clef=alto	F#, C#
	3 sostenidos	K:A clef=alto	F#, C#, G#
	4 sostenidos	K:E clef=alto	F#, C#, G#, D#
	5 sostenidos	K:B clef=alto	F#, C#, G#, D#, A#
	6 sostenidos	K:F# clef=alto	F#, C#, G#, D#, A#, E#
	7 sostenidos	K:C# clef=alto	F#, C#, G#, D#, A#, E#, B#

Otros Elementos del Lenguaje ABC

- **Ligaduras**: En ABC, las ligaduras se representan uniendo dos notas consecutivas con paréntesis, por ejemplo: (C D).
- **Puntillos**: Para representar el puntillo, se alarga la duración de la nota con fracciones. Ejemplo: C3/2 (nota Do con puntillo).